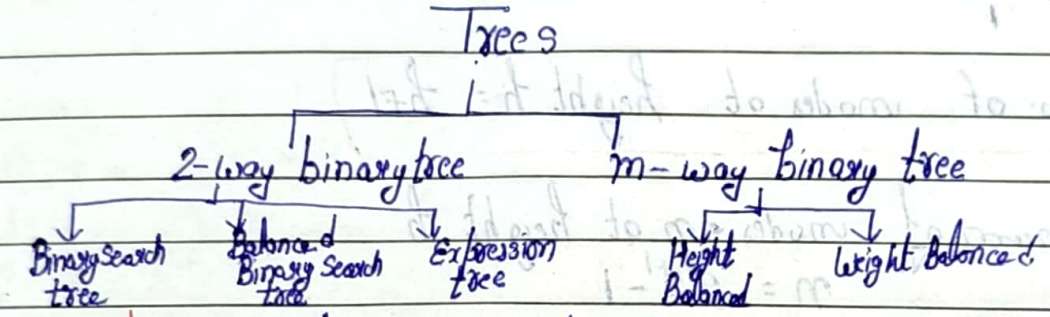
Date : / /
Page No.

Trees

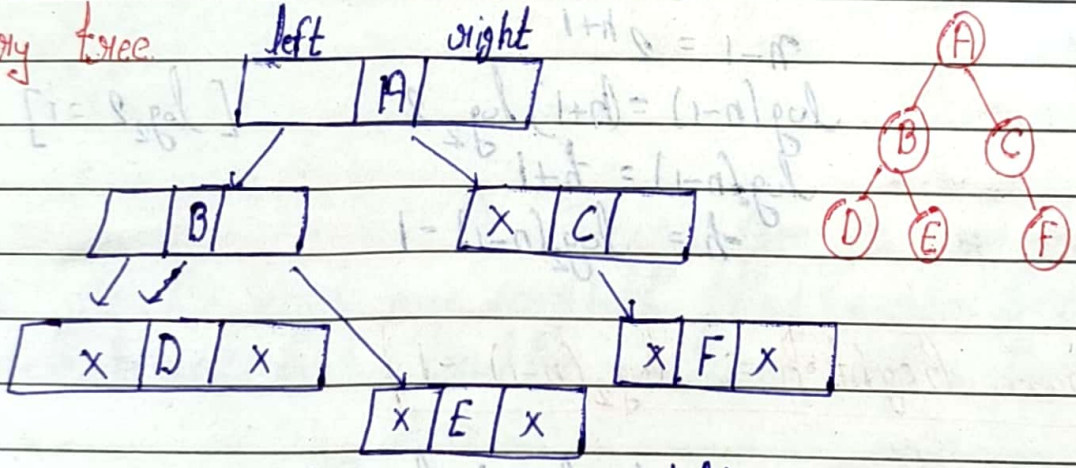
n nodes = (n-1) edges.

→ Depth of a node = level of a node

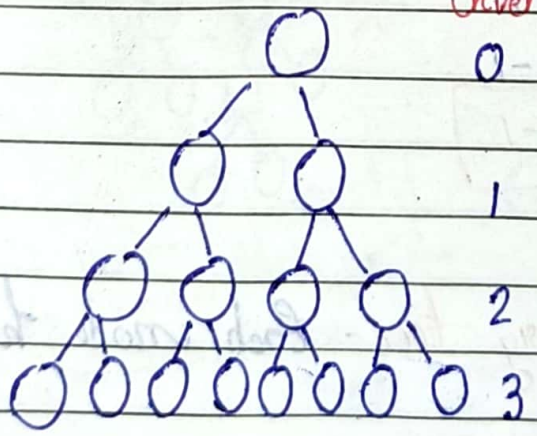
→ Height of a tree = level of a tree



Binary tree



- each node can have atmost 2 children.



level	node	
0	1	2^0
1	2	2^1
2	4	2^2
3	8	2^3

Max no. of node possible at any level ie = 2^i

Max no. of node at height 3 = 1+2+4+8
= 15.

Date : / /
Page No.

Max no. of nodes at height $h = 2^{h+1} - 1$

$= 2^0 + 2^1 + 2^2 + 2^3 \ldots 2^h$

$= \frac{2^{h+1} - 1}{1} = 2^{h+1} - 1$

Minimum no. of nodes at height $h = h+1$

Let maximum of nodes = n at height h

$n = 2^{h+1} - 1$

$n - 1 = 2^{h+1}$

$\log(n-1) = (h+1)\log_2 2 \quad [\log_2 2 = 1]$

$\log_2(n-1) = h+1$

$h = \log_2(n-1) - 1$

Minimum height of $h = \log_2(n-1) - 1$

Let Minimum no. of nodes at height $h = n$

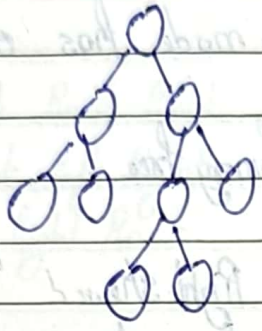
$n = h+1$

$h = n-1$

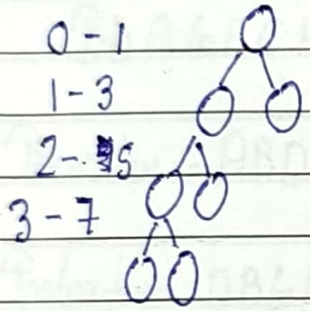
Maximum height $h = n-1$

- Full / Proper / strict binary tree - Each node have either 0 or 2 children.

No. of leaf node = no. of internal nodes + 1



Max nodes = $2^{h+1} - 1$

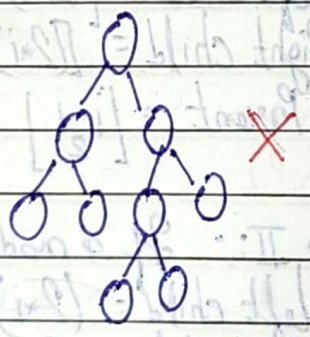


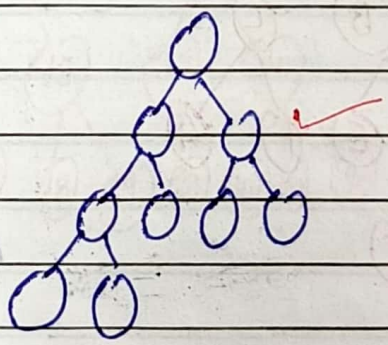
Min nodes = $2h + 1$

Min height = $\log_2[n+1] - 1$

Max height = $\frac{n-1}{2}$

- Complete Binary tree - A binary tree is complete binary tree all the levels are completely filled (except possibly the last level and last level has nodes as left as possible





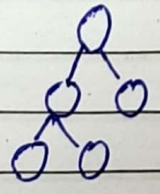
Max nodes = $2^{h+1} - 1$

Min nodes = 2^h

Max height = $\log n$

Min height = $\log_2(n+1) - 1$

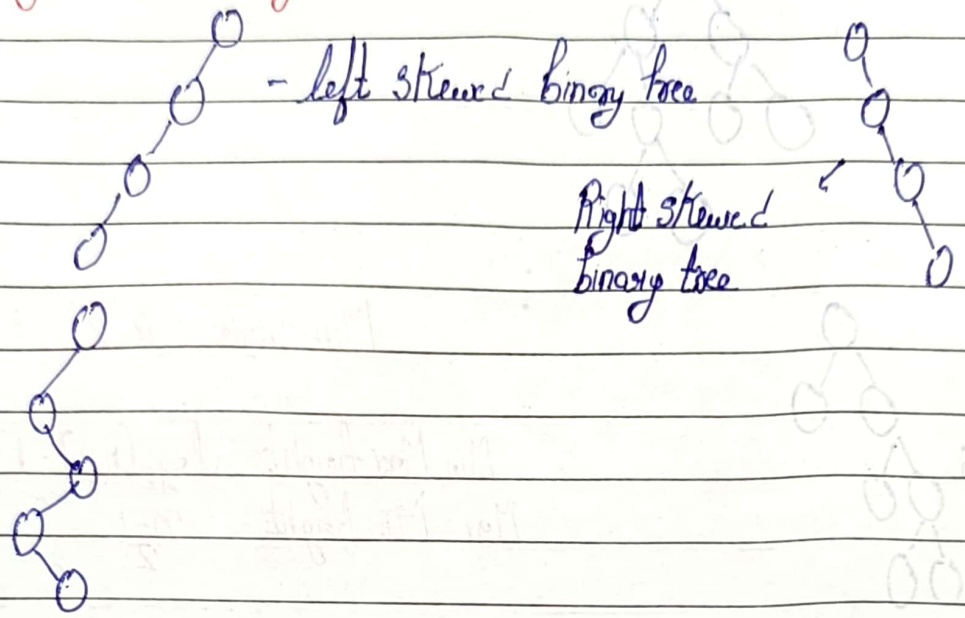
- Perfect binary tree - All internal nodes have 2 children and all leaves are at some level.



A perfect binary tree is full and completely binary tree but vice versa is not true.

Date : / /
Page No.

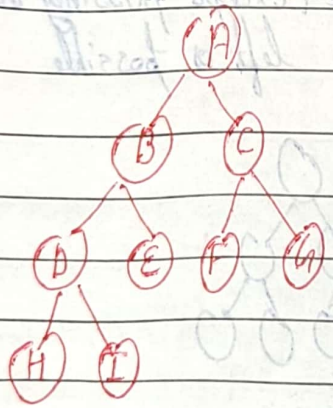
- Degenerate Binary tree - Each node has only one child.



- left skewed binary tree

Right skewed binary tree

Array representation of binary tree



Case I: If a node is at i^{th} index

→ left child = $[(2*i)+1]$

Right child = $[(2*i)+2]$

Parent = $\left[\frac{i-1}{2}\right]$

Case II: - If a node is at i^{th} index

Left child = $(2*i)$

Right child = $[(2*i)+1]$

Parent = $\left[\frac{i}{2}\right]$

Case I

A	B	C	D	E	F	G	H	I
0	1	2	3	4	5	6	7	8

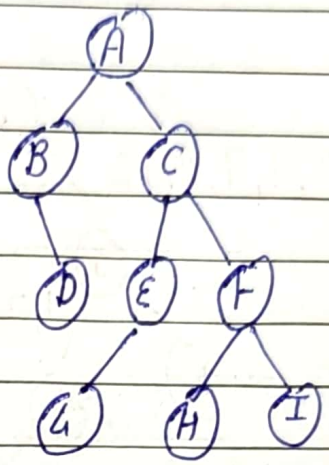
Case II

A	B	C	D	E	F	G	H	I
1	2	3	4	5	6	7	8	9

Date : / /
Page No.

Inorder, Preorder and Postorder traversal of a binary tree

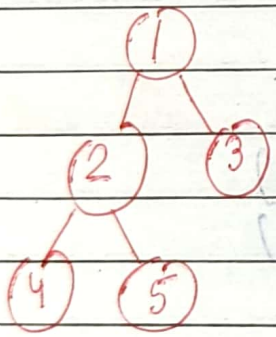
Inorder:- Left Root Right
Preorder- Root Left Right
Postorder- Left Right Root

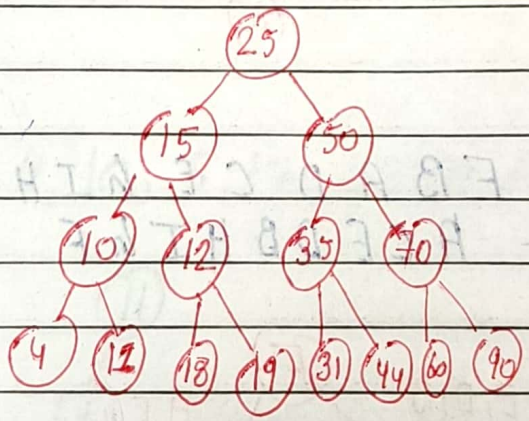


Inorder:-
BDAGECHFI

Preorder:- ABDCEGFHI

Postorder:- DBGEHIFCA



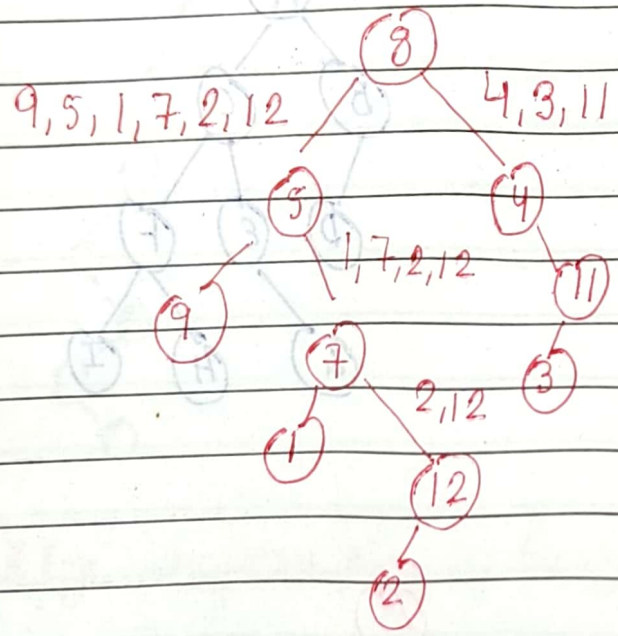


Preorder:- 1 2 4 5 3
Postorder:- 4 5 2 3 1
Inorder:- 4 2 5 1 3

Postorder:- 4 11 10 18 19 12 15 31 44 35 60 90 70 50 25
Preorder:- 25 15 10 4 11 12 18 19 50 35 31 44 70 60 90
Inorder:- 4 10 11 15 18 12 19 25 31 35 44 50 60 70 90

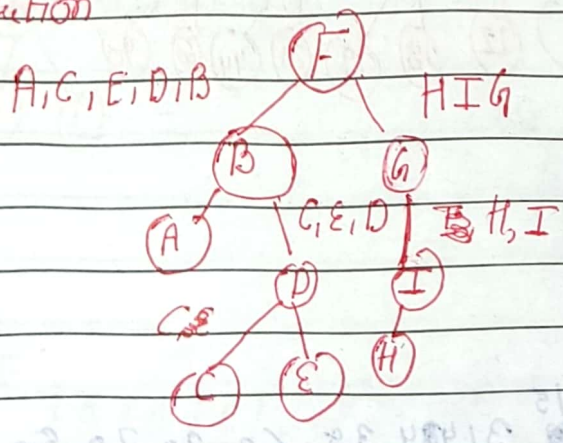
Date : / /
Page No.

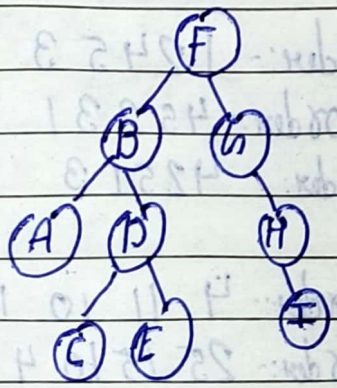
Postorder:- 9, 1, 2, 12, 7, 5, 3, 11, 4, 8 L R Root
Inorder - 9, 5, 1, 7, 2, 12, 8, 4, 3, 11 L Root R



Preorder:- F B A D C E G I H (Root L R)
Postorder - A C E D B H I G F (L R Root)

I solution

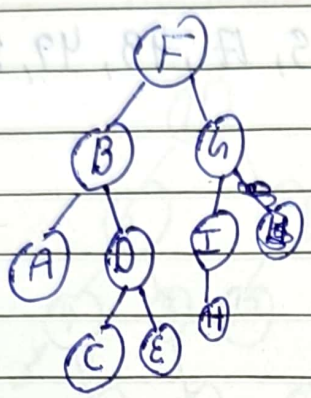




Preorder:- F B A D C E G I H
Postorder:- A C E D B H I G F

Date : / /
Page No.

II - Solution



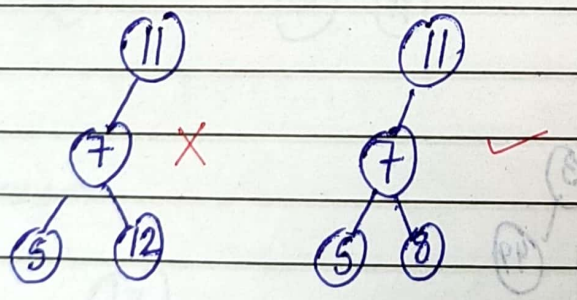
Using preorder and postorder unique full binary tree but not unique binary tree

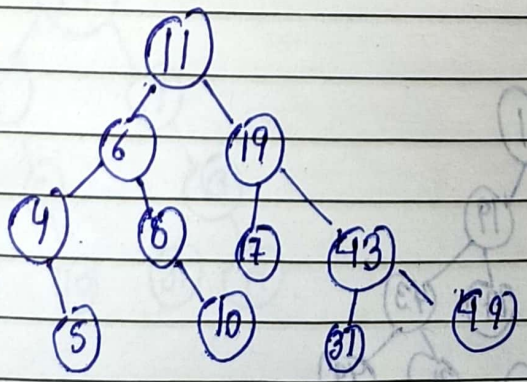
Binary Search Tree

(1) Each node must have atmost 2 children
(2) The value of left node must be smaller than parent node and value of right node must be greater than parents

Draw Binary Search tree

11, 6, 8, 19, 4, 10, 5, 17, 43, 49, 31



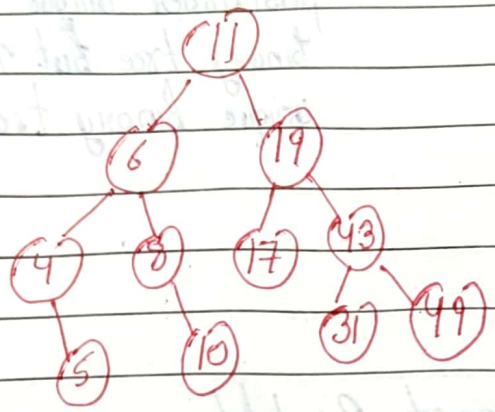


Date : / /
Page No.

Deletion in BST

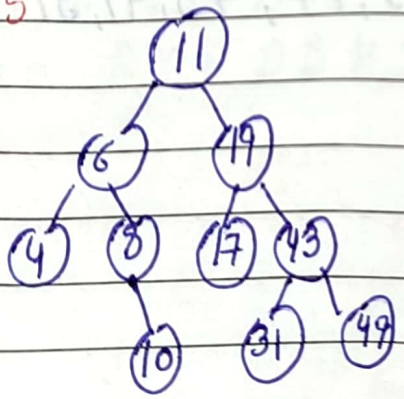
11, 6, 8, 19, 4, 10, 5, 17, 43, 49, 31

(1)

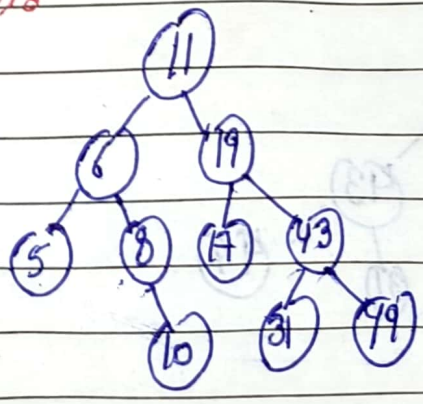


Case I No child
Case II 1 child
Case III 2 child

Case I No child
Delete 5

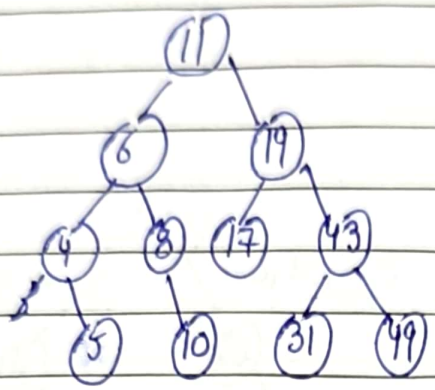


Case II 1 child
Delete 4

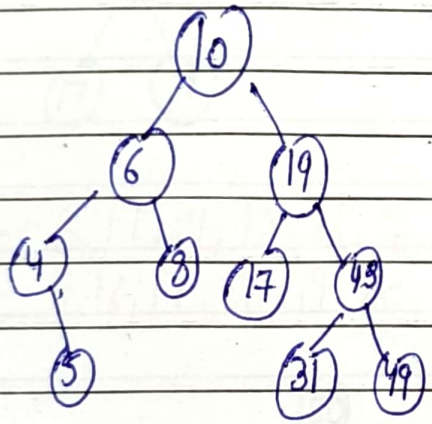


Case III 2 child (i) Inorder Predecessor (ii) Inorder Successor.

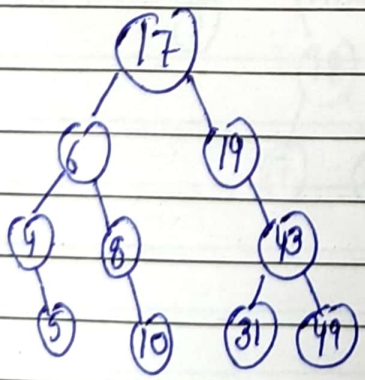
Delete (11)



Inorder Predecessor

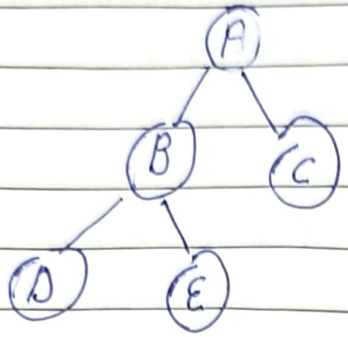


Inorder Successor.

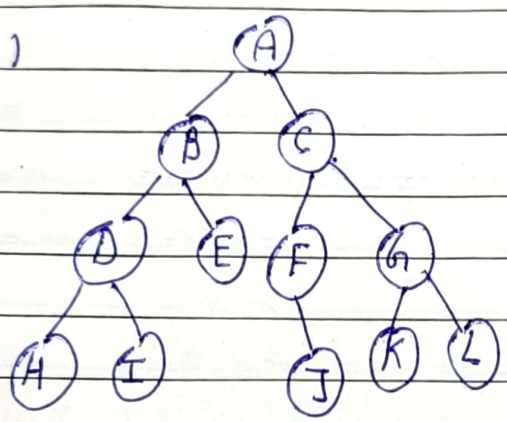


Date : / /
Page No.

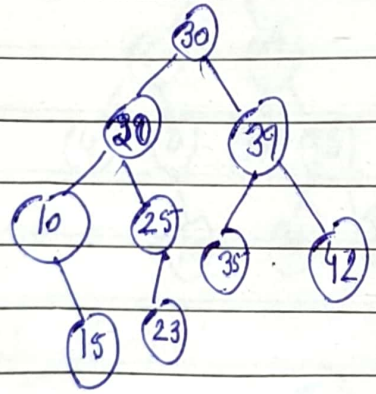
(1)



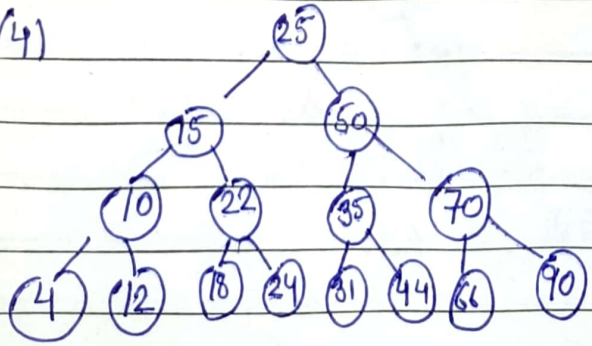
(2)



(3)



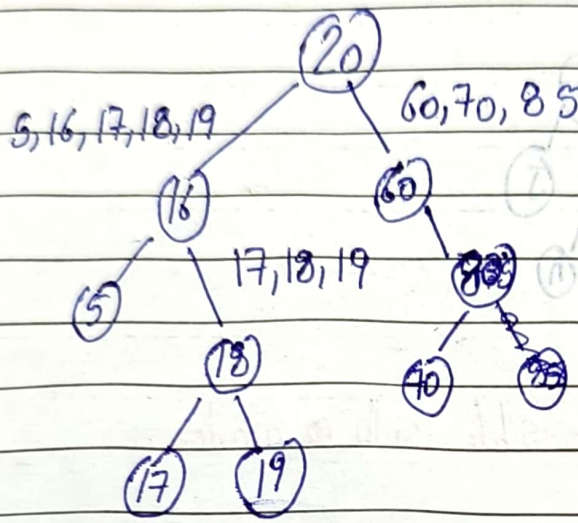
(4)



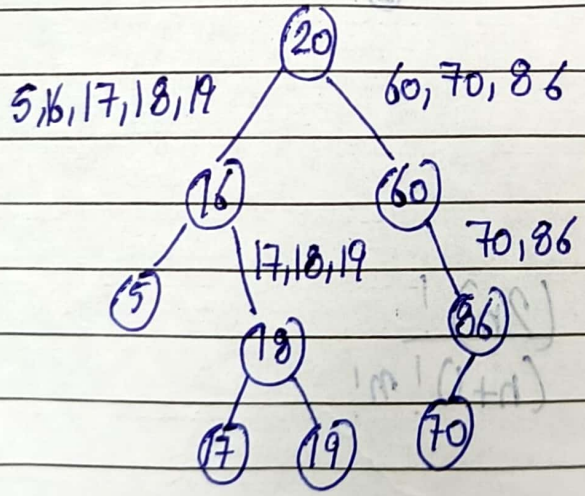
Date : / /
Page No.

Construction of BST when only preorder or postorder is given

Preorder :- 20, 16, 5, 18, 17, 19, 60, 85, 70 (Root left Right)
Inorder :- 5, 16, 17, 18, 19, 20, 60, 70, 85. (left Root Right)



Postorder :- 5, 17, 19, 18, 16, 70, 86, 60, 20 (left Right Root)
Inorder :- 5, 16, 17, 18, 19, 20, 60, 70, 86

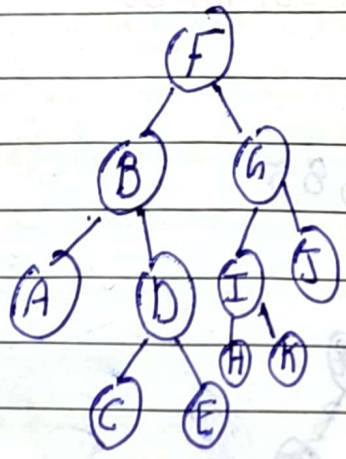


Date : / /
Page No.

Construct Binary tree from given preorder & postorder

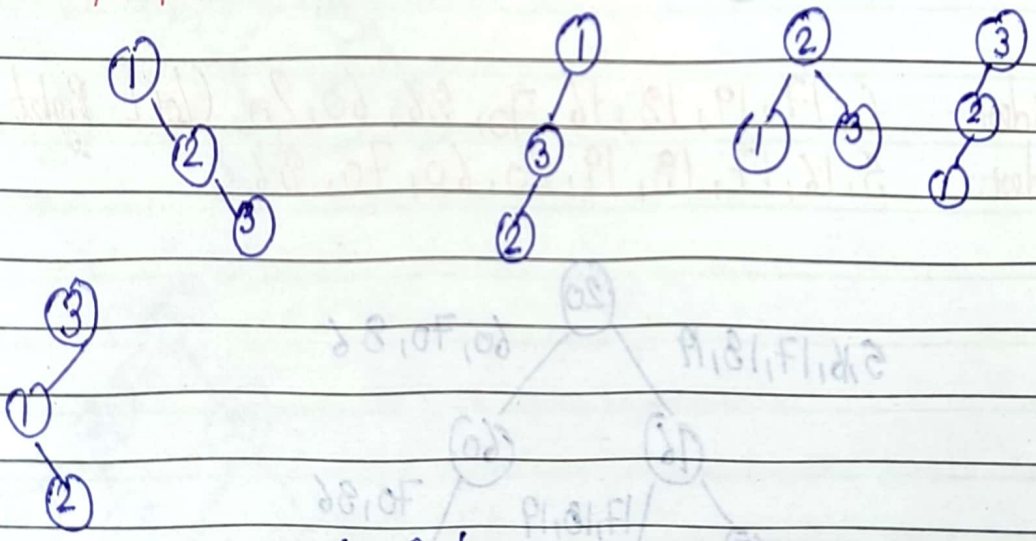
Preorder :- F B A D C E G I H K J (Root L R)

Postorder - A C E D B H K I J G F (L R Root)



Binary ~~tree~~ Search tree possible with 3 nodes.

1, 2, 3



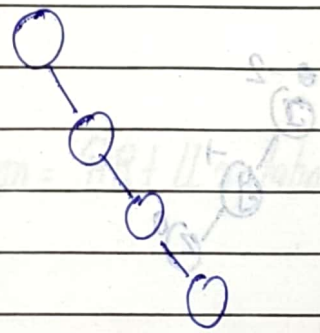
$$C_n = \frac{(2n)!}{(n+1)!\,n!}$$

Date : / /
Page No.

Time and space complexity
Binary Search tree

Operation	Worst case	Average case	Best case	Space
Search	O(n)	O(logn)	~~O(logn)~~ O(1)	O(n)
Insert	O(n)	O(logn)	~~O(logn)~~ O(1)	O(n)
Delete	O(n)	O(logn)	O(logn)	O(n)

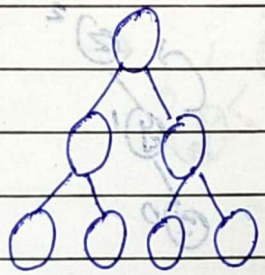
Worst case



$h = n-1$

Time complexity = O(n)

Average case



$n = 2^{h+1} - 1$

$n+1 = 2^{h+1}$

$\log_2(n+1) = (h+1)\log_2 2$

$h+1 = \log_2(n+1)$

$$\boxed{h = \log_2(n+1) - 1}$$